HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I

BÁO CÁO HẰNG TUẦN MÔN
THỰC TẬP CƠ SỞ

Giảng viên hướng dẫn : TS. Kim Ngọc Bách
Sinh viên thực hiện : Mã sinh viên
Nguyễn Việt Quang : B23DCCN691

Hà Nội – 2026

I. Thiết kế và thử nghiệm (Dry-run) Pipeline huấn luyện mô hình BLIP tiếng Việt

1. Mở đầu

Tiếp nối tiến độ của dự án "Finetune mô hình BLIP theo bộ dữ liệu tiếng Việt và xây dựng hệ thống mô tả thế giới thực cho người khiếm thị", sau khi hoàn tất công tác tiền xử lý và chuẩn hóa 61.239 mẫu của bộ dữ liệu UIT-OpenViIC vào tuần trước (04/04), bước trọng tâm tiếp theo là thiết kế một luồng huấn luyện (training pipeline) hoàn chỉnh. Về mặt bản chất, để mô hình học được ngôn ngữ mới (tiếng Việt), chúng ta cần chuyển đổi dữ liệu thô (ảnh và text) thành các ma trận số, cấu hình thuật toán tối ưu, và xây dựng một vòng lặp cập nhật trọng số (training loop). Tuy nhiên, rào cản lớn nhất trong giai đoạn này là giới hạn phần cứng, cụ thể là GPU NVIDIA GeForce GTX 1650 với chỉ 4GB VRAM. Huấn luyện các mô hình Vision-Language Model (VLM) lớn như BLIP trên cấu hình này rất dễ dẫn đến lỗi tràn bộ nhớ (Out Of Memory - OOM).

Do đó, nội dung thực hiện trong tuần này tập trung vào việc lập trình mã nguồn pipeline huấn luyện kết hợp chặt chẽ với các kỹ thuật tối ưu hóa phần cứng. Đồng thời, em tiến hành chạy thử nghiệm quy mô siêu nhỏ (dry-run/overfitting test) để kiểm chứng tính đúng đắn của logic mã nguồn trước khi tiến hành huấn luyện trên toàn bộ hơn 40.000 mẫu của tập train.

2. Mục tiêu thực hiện

Mục tiêu cụ thể của tuần này bao gồm bốn phần chính:

- **Phần thứ nhất:** Xây dựng lớp dữ liệu tùy chỉnh (UITViICDataset) để tự động đọc ảnh bằng OpenCV và mã hóa text bằng BlipProcessor.
- **Phần thứ hai:** Thiết lập hệ thống cung cấp dữ liệu (DataLoader) để phân lô (batching) dữ liệu truyền vào GPU một cách an toàn mà không gây lỗi VRAM.
- **Phần thứ ba:** Khởi tạo mô hình, thiết lập thuật toán tối ưu (Optimizer) và áp dụng chiến lược đóng băng tham số (Freeze) đối với khối Vision Encoder để giảm tải tính toán.
- **Phần thứ tư:** Thiết kế vòng lặp huấn luyện (Training Loop) tích hợp cơ chế cộng dồn đạo hàm (Gradient Accumulation) và chạy thử nghiệm trên một mini-dataset (train_tiny.csv) để quan sát sự hội tụ của hàm loss.

3. Môi trường và thiết lập thực hiện

Môi trường thực hiện trong tuần này sử dụng Python 3.11.9 trên hệ điều hành Windows 11, thông qua Jupyter Notebook (training_pipeline.ipynb). Các thư viện cốt lõi bao gồm:

- torch (PyTorch phiên bản hỗ trợ nhân CUDA) để quản lý tensor và tự động tính đạo hàm.
- transformers (Hugging Face) để nạp mô hình BLIP và processor.
- cv2 (OpenCV) và pandas để xử lý ảnh và đọc file dữ liệu CSV.
- tqdm để theo dõi tiến trình huấn luyện.

Thiết bị tính toán là GPU NVIDIA GeForce GTX 1650 (4GB VRAM).

4. Nội dung thực hiện chi tiết

Quá trình xây dựng pipeline được chia thành các phân hệ logic như sau:

4.1. Xây dựng Custom Dataset

Em thiết kế lớp UITViICDataset kế thừa từ torch.utils.data.Dataset.

Hàm __getitem__ thực hiện luồng công việc:

- Đọc đường dẫn và nạp ảnh bằng cv2.imread(), sau đó chuyển hệ màu từ BGR sang RGB để tương thích với BLIP.
- Đưa ảnh và caption tiếng Việt vào BlipProcessor để nhận về các tensor: pixel_values, input_ids, và attention_mask. Cấu hình padding="max_length" (50 token) và truncation=True được áp dụng.
- **Xử lý nhãn (Labels):** Tạo tensor labels bằng cách sao chép input_ids, sau đó thay thế các token đệm (pad_token_id = 0) thành giá trị -100. Kỹ thuật này rất quan trọng vì nó chỉ thị cho hàm loss của PyTorch bỏ qua các từ vô nghĩa khi tính toán cập nhật trọng số.

4.2. Thiết lập DataLoader và xử lý lỗi đa luồng (Multiprocessing)

DataLoader được khởi tạo với batch_size = 2 nhằm đảm bảo an toàn cho bộ nhớ 4GB VRAM. Trong quá trình triển khai, em gặp phải lỗi treo tiến trình (deadlock) khi thiết lập num_workers=2 trên môi trường Windows do cơ chế spawn của OS không tương thích tốt với Jupyter. Vấn đề đã được khắc phục bằng cách cấu hình num_workers=0, đẩy quá trình nạp dữ liệu về luồng chính (main thread).

4.3. Tải mô hình và áp dụng kỹ thuật Đóng băng (Freeze)

Mô hình Salesforce/blip-image-captioning-base được nạp lên GPU. Nhận thấy dữ liệu của UIT-OpenViIC chủ yếu là các vật thể và cảnh quan đời sống quen thuộc, khối trích xuất đặc trưng thị giác (Vision Encoder) của BLIP vốn đã biểu diễn rất tốt. Do đó, em tiến hành lặp qua toàn bộ tham số của model.vision_model và cấu hình requires_grad = False. Kỹ thuật này ngăn PyTorch lưu trữ gradient cho khối này, giúp tiết kiệm một lượng khổng lồ VRAM. Bộ tối ưu hóa được chọn là AdamW với learning rate 5e-5, và chỉ truyền vào những tham số chưa bị đóng băng (thuộc khối Text Decoder và Cross-Attention).

4.4. Xây dựng Training Loop với Gradient Accumulation

Do bị giới hạn ở mức batch_size=2, đồ thị đạo hàm sẽ rất nhiễu nếu cập nhật trọng số ngay. Để khắc phục, em cài đặt cơ chế Gradient Accumulation với biến ACCUMULATION_STEPS. Loss được chia nhỏ ở mỗi bước forward, và hàm optimizer.step() chỉ được gọi sau khi cộng dồn đủ số lượng mini-batch. Điều này giúp hệ thống mô phỏng được một effective batch_size lớn hơn (gấp 2, 4 hoặc 8 lần) mà vẫn an toàn cho VRAM.

5. Kết quả thực nghiệm (Dry-run Test)

Sau khi hoàn thiện pipeline, em tạo một tập dữ liệu siêu nhỏ train_tiny.csv (khoảng 10-20 mẫu) để chạy Overfitting Test trong 10 epochs. Việc này nhằm kiểm tra xem loss có thực sự giảm hay không trước khi tốn tài nguyên chạy trên toàn bộ tập dữ liệu.

Kết quả ghi nhận từ log của Notebook:

- **Về tối ưu tham số:** Tổng tham số của mô hình là ~223.9 triệu. Sau khi đóng băng Vision Encoder, số tham số cần huấn luyện giảm xuống còn ~137.8 triệu (chiếm khoảng 62%). Mức giảm gần 40% khối lượng tính toán này đã giúp GPU 4GB VRAM chạy mượt mà không gặp OOM.
- **Về sự hội tụ của hàm Loss:**
    - Epoch 1: loss = 4.4535
    - Epoch 2: loss = 3.1595
    - Epoch 5: loss = 1.8418
    - Epoch 10: loss = 0.3353

```
Training start...
Epoch: 1/10
Training: 100%|██████████| 9/9 [00:10<00:00,  1.15s/it, loss=4.4535]
Epoch: 2/10
Training: 100%|██████████| 9/9 [00:09<00:00,  1.10s/it, loss=3.1595]
Epoch: 3/10
Training: 100%|██████████| 9/9 [00:09<00:00,  1.02s/it, loss=2.8167]
Epoch: 4/10
Training: 100%|██████████| 9/9 [00:11<00:00,  1.23s/it, loss=2.4518]
Epoch: 5/10
Training: 100%|██████████| 9/9 [00:09<00:00,  1.08s/it, loss=1.8418]
Epoch: 6/10
Training: 100%|██████████| 9/9 [00:08<00:00,  1.00it/s, loss=1.5402]
Epoch: 7/10
Training: 100%|██████████| 9/9 [00:10<00:00,  1.22s/it, loss=0.9343]
Epoch: 8/10
Training: 100%|██████████| 9/9 [00:09<00:00,  1.08s/it, loss=0.7251]
Epoch: 9/10
Training: 100%|██████████| 9/9 [00:08<00:00,  1.00it/s, loss=0.4424]
Epoch: 10/10
Training: 100%|██████████| 9/9 [00:10<00:00,  1.21s/it, loss=0.3353]
```

Hình 5.1: Quá trình training thử nghiệm

Giá trị loss giảm ổn định và tiệm cận về gần 0 sau 10 vòng lặp. Điều này chứng minh rằng luồng truyền dữ liệu (forward pass), tính toán đạo hàm (backward pass) và xử lý nhãn padding (-100) đã hoạt động hoàn toàn chính xác.

6. Nhận xét và đánh giá

Quá trình xây dựng và chạy thử nghiệm pipeline trong tuần này đã giải quyết thành công bài toán tối ưu hóa tài nguyên phần cứng. Việc kết hợp chiến lược Freeze Vision Encoder và Gradient Accumulation là một quyết định kỹ thuật đúng đắn để vượt qua giới hạn của card đồ họa GTX 1650 4GB.

Kết quả dry-run với mức loss hội tụ cực tốt trên tập dữ liệu nhỏ (từ 4.45 xuống 0.33) là bằng chứng mạnh mẽ cho thấy hệ thống không có lỗi logic lập trình ngầm (bug), các tensor được gióng hàng (align) chuẩn xác và mô hình có năng lực học ngôn ngữ tiếng Việt từ dữ liệu đưa vào. Pipeline hiện tại đã đạt độ tin cậy cần thiết để tiến hành quá trình huấn luyện quy mô lớn.

7. Kết luận

Trong tuần này, em đã thiết kế, lập trình và thử nghiệm thành công pipeline huấn luyện cho mô hình BLIP. Mã nguồn xử lý tốt từ khâu đọc ảnh bằng OpenCV, tích hợp chuẩn

hóa nhãn của BlipProcessor, quản lý bộ nhớ GPU qua việc đóng băng tham số, cho đến thiết lập vòng lặp huấn luyện kết hợp cộng dồn đạo hàm.

Việc bài test dry-run thành công đánh dấu sự hoàn tất của khâu chuẩn bị kỹ thuật phức tạp nhất trong Giai đoạn 2. Tuần tiếp theo, em sẽ chính thức cấp toàn bộ tập dữ liệu 41.238 ảnh (train_cleaned.csv) vào pipeline này để tiến hành Full Finetuning, bắt đầu dạy cho mô hình cách mô tả môi trường thực tế bằng tiếng Việt.